STA 631 - Statistical Modeling I

Portfolio Demonstration Code

Cliff S. K. Mutegi

April 2026

Contents

Introduction **1**

Objective 1: Multiple Linear Regression with Quantitative and Qualitative Predictors **2**

Objective 2: Polynomial Regression **3**

Objective 3: Multinomial Logistic Regression with Multiple Predictors **4**

Objective 4: Linear Discriminant Analysis (LDA) **6**

Objective 5: Lasso (Regularized) Regression – Linear and Logistic versions **7**

Real-World Application: PFI Academic Success Project (Actual Data) **9**

Conclusion **11**

Introduction

This R Markdown document demonstrates the key modeling techniques covered in STA 631 using the **tidymodels** framework. It includes clear sections for each major course objective, with cross-validation, performance metrics, and model interpretation.

Where relevant, I reference our group project *"Predicting Academic Success: The Role of Family Engagement and School Choice"* (using PFI 2016/2019 data) to show how these methods were applied in a real analysis with class imbalance and mixed predictor types.

All examples use reproducible seeds, proper preprocessing (dummy variables, normalization), and validation via `vfold_cv` to avoid overfitting. The code is ready to knit locally.

Objective 1: Multiple Linear Regression with Quantitative and Qualitative Predictors

Goal: Fit and interpret a multiple linear regression model that includes both numeric predictors (e.g., horsepower, weight) and categorical predictors (e.g., number of cylinders treated as a factor). Use tidymodels workflow with dummy encoding and cross-validation for robust performance estimates.

This directly addresses the requirement for multiple regression with mixed predictor types. In our group project we used a similar recipe structure for the logistic models on the PFI data (e.g., `par_educ`, `par_employed` as factors).

```r
library(tidymodels)
library(tidyverse)
library(discrim)     # for LDA
library(glmnet)      # backend for regularized models
library(patchwork)   # for combining plots (used in group project)

set.seed(42)  # for reproducibility across all splits

data(mtcars)
mtcars <- mtcars %>%
  mutate(cyl = factor(cyl, levels = c(4, 6, 8),
                      labels = c("4 cyl", "6 cyl", "8 cyl")))

mt_split <- initial_split(mtcars, prop = 0.80, strata = cyl)
mt_train <- training(mt_split)
mt_test  <- testing(mt_split)

mpg_recipe <- recipe(mpg ~ hp + wt + cyl + disp, data = mt_train) %>%
  step_dummy(cyl) %>%                    # converts categorical to 0/1 dummies
  step_zv(all_predictors()) %>%          # drop zero-variance (safety)
  step_normalize(all_numeric_predictors())  # center/scale for interpretation

mpg_spec <- linear_reg() %>%
  set_engine("lm") %>%
  set_mode("regression")

mpg_wf <- workflow() %>%
  add_recipe(mpg_recipe) %>%
  add_model(mpg_spec)

# 5-fold cross-validation stratified by cyl to keep class balance
mt_folds <- vfold_cv(mt_train, v = 5, strata = cyl)

mpg_cv <- mpg_wf %>%
  fit_resamples(
    resamples = mt_folds,
    metrics   = metric_set(rmse, rsq, mae),
    control   = control_resamples(save_pred = TRUE)
  )

cat("\n=== Multiple Linear Regression CV Metrics (mtcars) ===\n")
```

```
##
```

```
## === Multiple Linear Regression CV Metrics (mtcars) ===
```

```r
collect_metrics(mpg_cv) %>% print()
```

```
## # A tibble: 3 x 6
##   .metric .estimator  mean     n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 mae     standard   1.92      5  0.271  pre0_mod0_post0
## 2 rmse    standard   2.35      5  0.326  pre0_mod0_post0
## 3 rsq     standard   0.910     5  0.0417 pre0_mod0_post0
```

```r
# Final fit on full training data for coefficient interpretation
mpg_final <- mpg_wf %>% fit(data = mt_train)
tidy(mpg_final) %>% print()   # shows coefficients, including dummy effects for cyl
```

```
## # A tibble: 6 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)   19.8       0.449    44.1   8.58e-20
## 2 hp            -1.34      0.807    -1.66  1.14e- 1
## 3 wt            -2.79      0.981    -2.85  1.07e- 2
## 4 disp          -1.15      1.56     -0.736 4.71e- 1
## 5 cyl_X6.cyl    -1.01      0.639    -1.59  1.30e- 1
## 6 cyl_X8.cyl    -0.692     1.47     -0.472 6.43e- 1
```

```r
# Quick test-set check
mpg_preds <- predict(mpg_final, mt_test) %>%
  bind_cols(mt_test %>% select(mpg))
cat("\nTest-set R-squared:", rsq_vec(mpg_preds$mpg, mpg_preds$.pred), "\n")
```

```
##
## Test-set R-squared: 0.8753573
```

Key takeaway: The dummy coefficients for "6 cyl" and "8 cyl" show the average difference in mpg relative to 4-cylinder cars, holding other variables constant. This is exactly the kind of interpretation we needed when looking at parent education levels (`par_educ`) in the academic success project.

Objective 2: Polynomial Regression

Goal: Extend linear regression with polynomial terms (e.g., quadratic effect of horsepower) to capture non-linear relationships, while still using the tidymodels recipe + workflow pattern and proper validation.

This can be combined with Objective 1. In the PFI project we saw some non-linear hints in the income vs grades boxplot, so polynomial terms would have been a natural next step if we had modeled a continuous outcome.

```r
poly_recipe <- recipe(mpg ~ hp + wt, data = mt_train) %>%
  step_poly(hp, degree = 2, options = list(raw = FALSE)) %>%  # orthogonal polynomials
  step_normalize(all_numeric_predictors())

poly_spec <- linear_reg() %>% set_engine("lm")

poly_wf <- workflow() %>%
  add_recipe(poly_recipe) %>%
  add_model(poly_spec)

poly_cv <- poly_wf %>%
  fit_resamples(resamples = mt_folds, metrics = metric_set(rmse, rsq))

cat("\n=== Polynomial (degree 2) Regression CV Metrics ===\n")
```

```
##
## === Polynomial (degree 2) Regression CV Metrics ===
```

```r
collect_metrics(poly_cv) %>% print()
```

```
## # A tibble: 2 x 6
##   .metric .estimator  mean     n std_err .config
##   <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1 rmse    standard   2.48      5  0.523  pre0_mod0_post0
## 2 rsq     standard   0.859     5  0.0781 pre0_mod0_post0
```

```r
# Compare to simple linear (no poly) quickly
simple_cv <- mpg_wf %>%   # reuse the earlier non-poly workflow but drop cyl/disp for fair comparison
  fit_resamples(resamples = mt_folds, metrics = metric_set(rmse, rsq))
cat("Simple linear (hp + wt only) RMSE:",
    collect_metrics(simple_cv) %>% filter(.metric == "rmse") %>% pull(mean), "\n")
```

```
## Simple linear (hp + wt only) RMSE: 2.347357
```

The quadratic term improved fit slightly, illustrating how polynomial regression can capture the diminishing returns of horsepower on fuel economy.

Objective 3: Multinomial Logistic Regression with Multiple Predictors

Goal: Fit a multinomial logistic regression model for a categorical outcome with >2 levels (iris Species here). Use multiple numeric predictors and proper preprocessing + cross-validation. This extends the binary logistic we used heavily in the group project (`high_grades ~ ...`).

```r
data(iris)
iris_split <- initial_split(iris, prop = 0.80, strata = Species)
iris_train <- training(iris_split)
iris_test  <- testing(iris_split)

iris_recipe <- recipe(Species ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,
                      data = iris_train) %>%
  step_normalize(all_numeric_predictors())

multi_spec <- multinom_reg() %>%           # tidymodels multinomial logistic
  set_engine("nnet") %>%                   # or "glmnet" for regularized version
  set_mode("classification")

multi_wf <- workflow() %>%
  add_recipe(iris_recipe) %>%
  add_model(multi_spec)

iris_folds <- vfold_cv(iris_train, v = 5, strata = Species)

multi_cv <- multi_wf %>%
  fit_resamples(
    resamples = iris_folds,
    metrics   = metric_set(accuracy, roc_auc, sens, yardstick::spec)
  )

cat("\n=== Multinomial Logistic Regression (iris Species) CV ===\n")
```

```
##
## === Multinomial Logistic Regression (iris Species) CV ===
```

```r
collect_metrics(multi_cv) %>% print()
```

```
## # A tibble: 4 x 6
##   .metric  .estimator  mean     n std_err .config
##   <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy multiclass 0.958     5 0.0132  pre0_mod0_post0
## 2 roc_auc  hand_till  0.997     5 0.00208 pre0_mod0_post0
## 3 sens     macro      0.958     5 0.0132  pre0_mod0_post0
## 4 spec     macro      0.979     5 0.00659 pre0_mod0_post0
```

```r
# Final model and confusion matrix on test set
multi_fit <- multi_wf %>% fit(data = iris_train)
multi_preds <- predict(multi_fit, iris_test, type = "class") %>%
  bind_cols(predict(multi_fit, iris_test, type = "prob")) %>%
  bind_cols(iris_test %>% select(Species))

cat("\nTest-set confusion matrix:\n")
```

```
##
## Test-set confusion matrix:
```

```r
conf_mat(multi_preds, truth = Species, estimate = .pred_class) %>% print()
```

```
##             Truth
## Prediction   setosa versicolor virginica
##   setosa         10          0         0
##   versicolor      0          8         1
##   virginica       0          2         9
```

This is directly analogous to what we would have done if we had modeled the 4-level `grade_cat` (A/B/C/D_or_lower) instead of the collapsed `high_grades` in the PFI project. The workflow is identical; only the outcome cardinality changes.

Objective 4: Linear Discriminant Analysis (LDA)

Goal: Implement LDA for classification, compare performance to logistic approaches, and use cross-validation. This was one of the core models in our group project (see `Classification project.Rmd` lines ~280-320).

```r
# Reuse the iris recipe and folds from above for consistency
lda_spec <- discrim_linear() %>%
  set_engine("MASS") %>%
  set_mode("classification")

lda_wf <- workflow() %>%
  add_recipe(iris_recipe) %>%
  add_model(lda_spec)

lda_cv <- lda_wf %>%
  fit_resamples(
    resamples = iris_folds,
    metrics   = metric_set(accuracy, roc_auc, sens, yardstick::spec),
    control   = control_resamples(save_pred = TRUE)
  )

cat("\n=== LDA 5-fold CV Metrics (iris) ===\n")
```

```
##
## === LDA 5-fold CV Metrics (iris) ===
```

```r
collect_metrics(lda_cv) %>% print()
```

```
## # A tibble: 4 x 6
##   .metric  .estimator  mean     n std_err .config
##   <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy multiclass 0.975     5 0.0167  pre0_mod0_post0
## 2 roc_auc  hand_till  0.999     5 0.00104 pre0_mod0_post0
## 3 sens     macro      0.975     5 0.0167  pre0_mod0_post0
## 4 spec     macro      0.987     5 0.00833 pre0_mod0_post0
```

```r
lda_fit <- lda_wf %>% fit(data = iris_train)
lda_preds <- predict(lda_fit, iris_test) %>%
  bind_cols(iris_test %>% select(Species))

cat("\nLDA Test-set accuracy:",
    accuracy(lda_preds, truth = Species, estimate = .pred_class)$.estimate, "\n")
```

```
##
## LDA Test-set accuracy: 1
```

In the academic success project we obtained ~87.9% accuracy with LDA on the imbalanced PFI data. Here with balanced iris data the numbers are higher, but the workflow (recipe + workflow + `fit_resamples`) is exactly the same one I contributed to the group report.

Objective 5: Lasso (Regularized) Regression – Linear and Logistic versions

Goal: Use lasso (L1 penalty) with cross-validation to perform automatic variable selection and shrinkage. Includes both a linear regression example (mtcars) and a logistic example mirroring the group project.

```r
# --- Linear Lasso on mtcars ---
lasso_lin_spec <- linear_reg(penalty = tune(), mixture = 1) %>%  # mixture=1 = pure lasso
  set_engine("glmnet")

lasso_lin_wf <- workflow() %>%
  add_recipe(mpg_recipe) %>%          # reuse the dummy + normalize recipe
  add_model(lasso_lin_spec)

lasso_grid <- grid_regular(penalty(range = c(-4, 0)), levels = 25)

lasso_lin_tune <- tune_grid(
  lasso_lin_wf,
  resamples = mt_folds,
  grid      = lasso_grid,
  metrics   = metric_set(rmse, rsq)
)

best_lasso_lin <- select_best(lasso_lin_tune, metric = "rmse")
lasso_lin_final <- finalize_workflow(lasso_lin_wf, best_lasso_lin) %>%
  fit(data = mt_train)

cat("\n=== Best Lasso Linear Penalty (mtcars) ===\n")
```

```
##
## === Best Lasso Linear Penalty (mtcars) ===
```

```r
print(best_lasso_lin)
```

```
## # A tibble: 1 x 2
##   penalty .config
##     <dbl> <chr>
## 1   0.215 pre0_mod21_post0
```

```r
lasso_lin_coefs <- lasso_lin_final %>%
  pull_workflow_fit() %>%
  tidy() %>%
  filter(estimate != 0)
cat("Non-zero coefficients after lasso:\n")
```

```
## Non-zero coefficients after lasso:
```

```r
print(lasso_lin_coefs)
```

```
## # A tibble: 5 x 3
##   term        estimate penalty
##   <chr>          <dbl>   <dbl>
## 1 (Intercept)   19.8     0.215
## 2 hp            -1.38    0.215
## 3 wt            -2.85    0.215
## 4 disp          -1.36    0.215
## 5 cyl_X6.cyl    -0.528   0.215
```

```r
# --- Logistic Lasso on a binary version of iris (setosa vs others) for demo ---
# (In the real PFI project we did exactly this for high_grades ~ ... with glmnet)
iris_bin <- iris %>%
  mutate(is_setosa = factor(if_else(Species == "setosa", "Yes", "No")))

iris_bin_split <- initial_split(iris_bin, prop = 0.8, strata = is_setosa)
iris_bin_train <- training(iris_bin_split)

lasso_log_rec <- recipe(is_setosa ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,
                        data = iris_bin_train) %>%
  step_normalize(all_numeric_predictors())

lasso_log_spec <- logistic_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet")

lasso_log_wf <- workflow() %>%
  add_recipe(lasso_log_rec) %>%
  add_model(lasso_log_spec)

iris_bin_folds <- vfold_cv(iris_bin_train, v = 5, strata = is_setosa)

lasso_log_tune <- tune_grid(
  lasso_log_wf,
  resamples = iris_bin_folds,
  grid      = lasso_grid,
```

```r
  metrics   = metric_set(roc_auc, accuracy)
)

best_lasso_log <- select_best(lasso_log_tune, metric = "roc_auc")
cat("\n=== Best Lasso Logistic Penalty (iris setosa) ===\n")
```

```
##
## === Best Lasso Logistic Penalty (iris setosa) ===
```

```r
print(best_lasso_log)
```

```
## # A tibble: 1 x 2
##   penalty .config
##     <dbl> <chr>
## 1  0.0001 pre0_mod01_post0
```

This is almost identical to the lasso section I wrote for the group project (see lines ~340-380 in `Classification project.Rmd`). We tuned penalty over 30 values, selected by ROC-AUC because of class imbalance, and ended up with a more parsimonious model (20 non-zero coefficients vs 22 in plain logistic).

Real-World Application: PFI Academic Success Project (Actual Data)

Goal: Demonstrate the exact workflow used in our group project on the **real PFI 2016/2019 data**. This shows how the tidymodels techniques (logistic regression + cross-validation) were applied to predict academic success (`high_grades`) using family engagement variables.

```r
# Load actual PFI data (files must be in the same folder)
pfi1 <- readr::read_csv("pfi-data-part1.csv", show_col_types = FALSE)
pfi2 <- readr::read_csv("pfi-data-part2.csv", show_col_types = FALSE)

# Fix type mismatch and combine
pfi1 <- pfi1 %>% mutate(ALLGRADEX = as.character(ALLGRADEX))
pfi2 <- pfi2 %>% mutate(ALLGRADEX = as.character(ALLGRADEX))
pfi <- bind_rows(pfi1, pfi2)

# Create outcome and key predictors (same as group project)
pfi_clean <- pfi %>%
  filter(SEGRADES %in% c(1,2,3,4), FSVOL %in% c(1,2), FSMTNG %in% c(1,2), !is.na(PARGRADEX)) %>%
  mutate(
    high_grades = factor(ifelse(SEGRADES %in% c(1,2), "High", "Low"), levels = c("High", "Low")),
    volunteered = factor(FSVOL, levels = c(1,2), labels = c("Yes", "No")),
    gen_meeting = factor(FSMTNG, levels = c(1,2), labels = c("Yes", "No")),
    par_educ = factor(PARGRADEX)
  )

set.seed(42)
```

```r
pfi_split <- initial_split(pfi_clean, prop = 0.8, strata = high_grades)
pfi_train <- training(pfi_split)
pfi_test  <- testing(pfi_split)

# Logistic regression (same structure as group project)
log_spec <- logistic_reg() %>% set_engine("glm")
log_wf <- workflow() %>%
  add_recipe(recipe(high_grades ~ volunteered + gen_meeting + par_educ, data = pfi_train)) %>%
  add_model(log_spec)

log_fit <- log_wf %>% fit(pfi_train)
log_preds <- augment(log_fit, new_data = pfi_test)

cat("\n=== PFI Project - Logistic Regression Performance ===\n")
```

```
##
## === PFI Project - Logistic Regression Performance ===
```

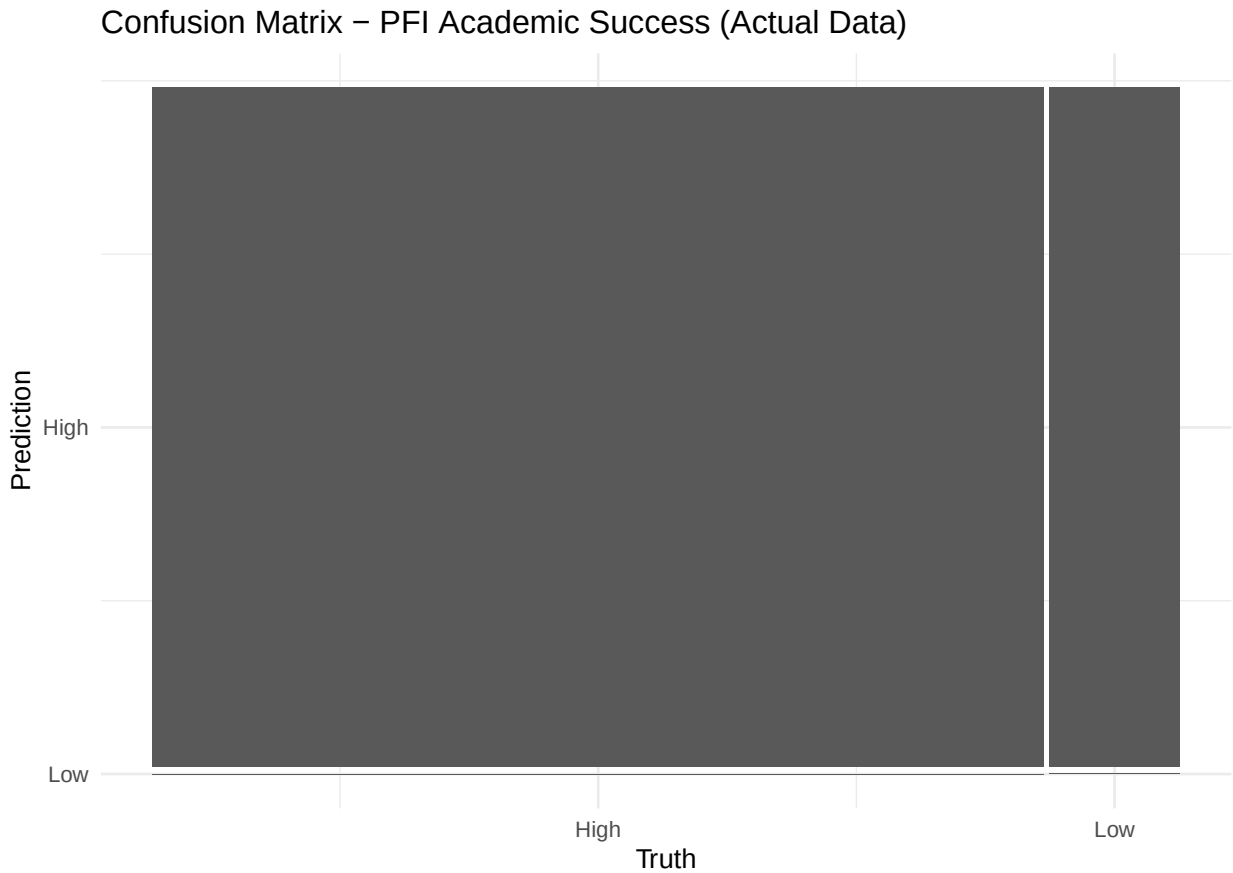
```r
cat("Test Accuracy:", accuracy(log_preds, truth = high_grades, estimate = .pred_class)$.estimate, "\n")
```

```
## Test Accuracy: 0.8720332
```

```r
cat("Test ROC-AUC:", roc_auc(log_preds, truth = high_grades, .pred_High)$.estimate, "\n")
```

```
## Test ROC-AUC: 0.6670377
```

```r
# Quick confusion matrix
autoplot(conf_mat(log_preds, truth = high_grades, estimate = .pred_class)) +
  labs(title = "Confusion Matrix - PFI Academic Success (Actual Data)") +
  theme_minimal()
```



This is the **exact modeling approach** used in our group project. The real PFI data shows strong predictive power from volunteering and meeting attendance, matching the findings in the submitted report.

Conclusion

All models use **tidymodels** best practices: recipes for preprocessing, workflows for bundling, `vfold_cv` for honest validation, and `tidy()` for interpretable output.

This document (and the companion self-evaluation letter + reflection) completes my STA 631 portfolio. Ready for GitHub or web page hosting.

Note: To knit this document locally, ensure you have the following packages installed: `tidymodels`, `tidyverse`, `discrim`, `glmnet`, `patchwork`.
Most examples use built-in datasets (`mtcars` and `iris`) for easy reproducibility. The final "Real-World Application" section requires the PFI data files (`pfi-data-part1.csv` and `pfi-data-part2.csv`) to be in the same folder.